

HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

SCHOOL OF INFORMATION AND COMMUNICATION TECHNOLOGY



CAPSTONE PROJECT

TOPIC 3

Tree Data Structure Visualization

Students: GROUP 14

1. Trần Quang Thắng - 202417196
2. Nguyễn Tiên Dũng - 202417116
3. Nguyễn Khắc Khiêm - 202417142
4. Vũ Khánh Toàn - 202417204
5. Nguyễn Xuân Thủy - 202417202

Class: 166241

Supervisor: Dr. Nguyễn Thị Thu Trang

Hanoi, June 23th, 2026

TABLE OF CONTENTS

LIST OF GROUP MEMBERS & TASK ASSIGNMENT	i
1. INTRODUCTION	1
1.1 Problem Statement	1
1.2 Project Objective	1
1.2.1 Using Object-Oriented Programming (OOP)	2
1.2.2 Implementation of the MVC Architecture	2
1.2.3 Interactive Visualization and Algorithm Animation	2
1.3 Scope of Project	3
2. BACKGROUND KNOWLEDGE	4
2.1 Overview of Tree Data Structures	4
2.2 Supported Tree Types	5
2.2.1 General Tree	5
2.2.2 Binary Tree	5
2.2.3 Binary Search Tree (BST)	6
2.2.4 AVL Tree	7
2.2.5 Red-Black Tree	8
2.3 Tree Operations and Traversal	9
2.3.1 Insertion	9
2.3.2 Deletion	9
2.3.3 Searching	10
2.3.4 Updating	10
2.3.5 Traversal	10
3. CORE TECHNOLOGIES	11

4. REQUIREMENT ANALYSIS	12
4.1 Core Logic	12
4.2 Interactions	12
4.3 User Interface	13
4.4 Object-Oriented Programming Requirements	13
5. SYSTEM DESIGN & MVC ARCHITECTURE	14
5.1 Use Case Modelling	14
5.1.1 System Actor	15
5.1.2 General Functions	15
5.2 Class Diagram	16
5.2.1 General Class Diagram	16
5.2.2 Model Class Diagram	17
5.2.3 View Class Diagram	18
5.2.4 Controller Class Diagram	19
5.2.5 Theme Class Diagram	21
5.2.6 Util Class Diagram	22
5.3 MVC Architecture	23
5.4 Exception Handling	24
5.4.1 Custom Domain Exceptions	24
5.4.2 Defensive Programming via Standard Exceptions	25
5.4.3 UI Input Validation and JavaFX Integration	26
6. DEMONSTRATION & RESULTS	27
6.1 System Overview and Navigation	27
6.2 Core Functionalities Demonstration	28

6.2.1 Dynamic Tree Operations (Insert, Delete, Search)	28
6.2.2 Algorithmic Tracking via Pseudo-code	29
6.2.3 Tree Traversals	30
6.3 Advanced GUI Features	31
6.3.1 Infinite Canvas and Camera Controls	31
6.3.2 Undo/Redo & History Management	31
6.3.3 Premium Dark/Light Themes	32
6.4 Evaluation and Results	33
7. CONCLUSION	34

LIST OF GROUP MEMBERS & TASK ASSIGNMENT

Project Title: Tree Data Structure Visualization

Course: Object-Oriented Programming

No.	Student Name	Student ID	Assigned Tasks	Evaluation
1	Trần Quang Thắng	202417196	Model architecture design, tree data structure algorithm implementation, pseudo-code display implementation, controller connection support, report writing support, and use case diagram design	Very Good
2	Nguyễn Tiến Dũng	202417116	Controller development, view designing and building, bug fixing, refactoring, navigation flow implementation, design class diagram, and use case diagram support	Very Good
3	Nguyễn Khắc Khiêm	202417142	Developed controller logic, designed and built view components, supported animation handling, implemented navigation flow, refactored code, and contributed to class diagram and use case diagram design.	Very Good
4	Vũ Khánh Toàn	202417204	Model diagram design, BST, AVL and Red-Black Tree model implementation, pseudo-code display support, controller connection support, debugging and testing, main report and slide preparation, and partial controller/view development	Very Good
5	Nguyễn Xuân Thủy	202417202	Partial View and Controller development, exception handling implementation, and input error handling support.	Very Good

Remarks:

All group members actively participated in the project and completed their assigned tasks on schedule. The group worked collaboratively to ensure the successful completion of the capstone project.

1. Introduction

1.1. Problem Statement

Data structures are fundamental to computer science and software engineering, and tree data structures play an especially important role. Trees support the efficient organization of hierarchical data and operations such as searching, insertion, deletion, and traversal, and are widely applied in file systems, databases, and network routing systems.

Despite their importance, tree data structures remain challenging for many students and beginner programmers. While the theoretical concepts are generally accessible, visualizing how operations affect the relationships between nodes — and how the structure changes step by step — is considerably harder to grasp from static materials alone.

Traditional learning resources such as diagrams, textbook illustrations, and code examples can explain tree behavior, but they cannot demonstrate it dynamically. As a result, learners are often left to mentally simulate operations rather than observe them directly, which creates a gap between theoretical understanding and practical intuition.

This project addresses that gap by introducing an interactive, JavaFX-based visualization tool for tree data structures. The application renders trees graphically and animates common operations in real time, enabling users to observe structural changes directly through visualization. By turning abstract concepts into interactive visuals, the tool aims to improve learning effectiveness and user engagement. The project also serves as a practical application of **Object-Oriented Programming (OOP)** principles and the **Model–View–Controller (MVC) architectural** pattern, promoting a clean, maintainable, and well-structured software design.

1.2 Project objective

The primary goal of this project is to develop a highly interactive tree data structure visualization application that helps users intuitively understand how tree algorithms operate. The application achieves this through step-by-step animations and real-time graphical feedback, making abstract operations tangible and accessible to learners at all levels.

1.2.1. Using Object-Oriented Programming (OOP) Principles

This project applies the four core principles of Object-Oriented Programming to design a clean, maintainable, and extensible tree visualization system.

- **Encapsulation:** Each node and tree class conceals its internal state from external components. For example, the `value` field in `Node` is declared `protected` and accessed only through dedicated getter and setter methods. Similarly, the `root` reference in each tree class is not directly exposed to controllers or the UI layer, ensuring the tree's internal structure is protected from unintended modification.

- **Abstraction:** The project defines an abstract class `AbstractTree` that declares the common interface all tree types must implement — including `insert()`, `delete()`, `search()`, `update()`, and `traverse()` — without prescribing how each operation is carried out. Concrete implementations are delegated to subclasses such as `GeneralTree`, `BinarySearchTree`, `AVLTree`, and `RedBlackTree`, each of which provides behavior suited to its own structural rules.
- **Inheritance:** Tree classes are organized in a multi-level hierarchy that mirrors the natural relationships between tree types. `BinarySearchTree` extends `BinaryTree`, and both `AVLTree` and `RedBlackTree` extend `BinarySearchTree`. This structure allows shared logic to be written once and reused across subclasses, reducing duplication and simplifying future maintenance.
- **Polymorphism:** The application interacts with all tree types through the common `AbstractTree` interface. Regardless of whether the active tree is a BST, an AVL Tree, or a Red-Black Tree, the same method calls — `insert()`, `delete()`, `traverse()` — are dispatched uniformly. Each subclass overrides these methods to produce behavior appropriate to its structure, allowing the system to support multiple tree types without altering the controller logic.

1.2.2. Implementation of the MVC architecture

The application is structured using the **Model–View–Controller (MVC) architectural pattern**, dividing the system into three independent layers to keep the codebase clean, organized, and easy to extend.

- The **Model** manages all tree data structures, algorithm logic, operation history, and animation step generation.
- The **View** is responsible for rendering the graphical interface and displaying tree visualizations to the user.
- The **Controller** handles user interactions and coordinates communication between the Model and the View.

This separation of concerns ensures that changes to one layer do not affect the others, improving maintainability and scalability.

1.2.3 Interactive Visualization and Algorithm Animation

The application visualizes tree operations through real-time graphical animation, covering key aspects such as:

- How each algorithm step — including comparisons, insertions, deletions, and rotations — is broken down and displayed one action at a time.
- How the tree structure updates visually on the canvas after every operation.

- How the pseudocode panel highlights the corresponding step in sync with each animation frame.
- How users can control the playback speed, pause, and resume the animation to observe the algorithm at their own pace.

By animating these processes, the project enables users to directly observe how tree operations affect the underlying structure in real time, providing both an educational understanding of tree algorithms and a practical demonstration of interactive software design.

1.3. Scope of Project

This project focuses on the visualization and simulation of common tree data structures for educational purposes. The following tree types are supported:

- **General Tree:**
 - Visualization of hierarchical parent-child relationships.
 - Support for basic structure display and tree traversal.
- **Binary Tree:**
 - Visualization of binary tree structures.
 - Support for insertion and traversal operations.
- **Binary Search Tree (BST):**
 - Visualization of insertion, deletion, searching, and traversal operations.
 - Demonstration of BST ordering properties.
- **AVL Tree:**
 - Visualization of self-balancing behavior during insertion and deletion.
 - Demonstration of single and double rotations with step-by-step animation.
- **Red-Black Tree:**
 - Visualization of balancing mechanisms through node recoloring and rotations.
 - Demonstration of Red-Black Tree properties and how they are maintained after each operation.

Project Boundaries

The project is strictly scoped to educational visualization and interactive simulation of tree operations. Applications such as database indexing, file system management, and large-scale data processing fall outside the boundaries of this project.

2. Background knowledge

2.1. Overview of Tree Data Structures

A tree is a non-linear, hierarchical data structure consisting of a collection of nodes connected by edges to represent parent-child relationships. Unlike linear structures such as arrays, linked lists, or stacks, trees organize data across multiple levels in a top-down hierarchy. To fully understand tree algorithms and their corresponding visualizations, the following fundamental terminology must be defined:

Node: The basic building block of a tree. Each node holds a value and maintains references to its child nodes.

Root: The topmost node in the tree hierarchy, uniquely identified by having no parent. All tree operations and traversals originate from the root.

Edge: A directed link connecting two nodes, representing the relationship between them. A valid tree with N nodes contains exactly $N - 1$ edges.

Parent, Child, and Sibling:

- **Parent:** A node that has one or more child nodes branching beneath it.
- **Child:** A node directly connected below a parent node.
- **Sibling:** Nodes that share the same parent node.

Leaf (External Node): A node with no children, representing the terminal endpoints of the tree.

Depth (of a Node): The level of a node in the tree, measured as the number of nodes along the path from the root to that specific node (or the number of edges). Depending on the convention, the root typically has a depth of 0 or 1.

Height (of a Node): Evaluated as the number of nodes on the longest downward path from that node to a leaf. In our project's implementation, **a leaf node has a height of 1**, and an empty subtree has a height of 0. The height of the tree is defined as the height of its root node.

Subtree: A node within a tree together with all of its descendants, forming a smaller tree structure contained within the larger one.

Balance Factor: A metric used in self-balancing trees to assess whether a node is unbalanced, typically calculated as the difference in height between a node's left and right subtrees. It is a central concept in AVL Trees.

Rotation: A structural operation used in self-balancing trees (AVL and Red-Black Trees) to restore balance after insertion or deletion. Rotations reorganize node pointers without altering the in-order sequence of the tree. Common forms include Left Rotation and Right Rotation.

Node Color: A property exclusive to Red-Black Trees, where each node is assigned a color — either red or black — stored as an additional bit. This property, combined with rotation operations, ensures the tree remains approximately balanced after every modification.

2.2. Tree Types

This project implements and visualizes five fundamental tree data structures, each representing a distinct level of complexity and a different set of structural properties.

2.2.1. General Tree

A General Tree is the most unrestricted form of tree data structure, in which each node may have an arbitrary number of children. Because no constraint is placed on the branching factor, it is the most flexible tree model for representing hierarchical relationships such as organizational charts, file directory structures, and XML document hierarchies.

In this project, each General Tree node stores a list of references to its children. Supported operations include insertion under a specified parent, deletion, value updating, searching, and traversal via Pre-order, Post-order, and Breadth-First Search (BFS).

2.2.2. Binary Tree

A Binary Tree is a tree structure in which each node has at most two children, referred to as the left child and the right child. Unlike a Binary Search Tree, a standard Binary Tree imposes no ordering constraint on node values, meaning the left or right subtree may contain values greater or smaller than the parent without restriction.

In this project, each Binary Tree node maintains two pointers: left and right. Supported operations include insertion under a specified parent, deletion, searching, and traversal via In-order, Pre-order, Post-order, and BFS.

2.2.3. Binary Search Tree (BST)

A Binary Search Tree is a Binary Tree that enforces a strict ordering property: for any given node, all values in its left subtree are strictly smaller and all values in its right subtree are strictly greater. This property enables considerably more efficient searching, insertion, and deletion compared to an unordered Binary Tree, with an average time complexity of $O(\log n)$.

However, a standard BST does not guarantee balance. When elements are inserted in sorted order, the tree degenerates into a linear chain, reducing all operations to $O(n)$ time complexity in the worst case.

In this project, the BST extends the Binary Tree and overrides the `insert()`, `delete()`, `search()`, and `update()` methods to enforce the ordering property throughout all operations.

2.2.4. AVL Tree

An AVL Tree (named after its inventors Adelson-Velsky and Landis) is a self-balancing Binary Search Tree that maintains an additional Balance Factor for each node, defined as:

Balance Factor = Height(Left Subtree) – Height(Right Subtree)

The AVL property requires that the Balance Factor of every node remain within $\{-1, 0, +1\}$ at all times. Whenever an insertion or deletion violates this constraint, the tree restores balance through one of four rotation operations: Left Rotation (LL), Right Rotation (RR), Left-Right Rotation (LR), or Right-Left Rotation (RL). This balance invariant guarantees $O(\log n)$ worst-case time complexity for all operations.

In this project, the AVL Tree extends the BST and introduces a `rebalance()` method, which is invoked after every structural modification to evaluate the Balance Factor and apply the appropriate rotation.

2.2.5. Red-Black Tree

A Red-Black Tree is a self-balancing Binary Search Tree that maintains approximate balance through a set of color-based structural rules rather than strict height balancing. Each node is assigned a color — either Red or Black. A valid Red-Black Tree must satisfy the following properties at all times:

- Every node is either red or black.
- The root is always black.
- Every null leaf is considered black.
- No two consecutive red nodes may exist — a red node's parent and children must both be black.
- Every path from any node to its descendant null leaves contains the same number of black nodes (the Black-Height property).

These properties collectively ensure that the longest path from root to leaf is at most twice the length of the shortest, guaranteeing $O(\log n)$ time complexity for insertion, deletion, and searching.

In this project, the Red-Black Tree extends the BST and implements full rebalancing logic through a combination of node recoloring and rotation operations after every structural modification.

2.3. Tree Operations & Traversal

Tree data structures support a well-defined set of operations that allow users to create, modify, search, and explore the structure of a tree. This project implements the following fundamental operations for all supported tree types.

2.3.1. Insertion

Insertion is the operation used to add a new node to the tree. The behavior of this operation depends on the type of tree.

In a General Tree and a Binary Tree, the new node is inserted as a child of a user-specified parent node. In a Binary Search Tree (BST), the new node is placed automatically according to the ordering property: values smaller than the current node are directed to the left subtree, while larger values are directed to the right, until an empty position is found. In an AVL Tree and a Red-Black Tree, insertion follows the same BST-based logic but is subsequently followed by a rebalancing process to preserve the structural invariants of the tree.

2.3.2. Deletion

Deletion is the operation used to remove a node from the tree. Its behavior varies significantly depending on the tree type.

In a General Tree and a Binary Tree, deleting a node also removes its entire subtree. The deleted node and all of its descendants are discarded together, as there is no reattachment mechanism for unordered trees.

In a Binary Search Tree, deletion handles three distinct cases. If the target node has no children, it is removed directly. If it has one child, it is replaced by that child. If it has two children, it is replaced by its in-order successor — the smallest node in its right subtree — which is then removed from its original position, thereby preserving the ordering property.

In an AVL Tree, deletion follows the same BST process but triggers a `rebalance()` call at each recursive return step to restore the balance factor of every affected ancestor node.

In a Red-Black Tree, deletion is handled through a `transplant()` operation that directly rewires parent pointers. After removal, if the deleted node was Black, a `fixDelete()` procedure is activated to restore the Red-Black properties through a series of recoloring and rotation cases.

2.3.3. Searching

Searching is used to determine whether a node with a given value exists in the tree.

In a General Tree and a Binary Tree, the search process must examine the subtrees recursively because there is no ordering rule to guide traversal. In a BST, AVL Tree, and Red-Black Tree, the ordering property allows the search to eliminate one subtree at each step, resulting in more efficient performance and an average time complexity of $O(\log n)$.

2.3.4. Updating

Updating is the operation used to modify the value of an existing node.

In a General Tree and a Binary Tree, the target node is located and its value is replaced directly. In ordered trees such as BSTs, AVL Trees, and Red-Black Trees, updating is implemented as a delete-and-insert operation to ensure that the strict ordering and balance properties remain valid after the modification.

2.3.5. Traversal

Traversal refers to the process of visiting every node in the tree in a specific order and recording the sequence of visited values. This project supports four traversal strategies.

- **Pre-order (Root → Left → Right):** The current node is visited before its children. This traversal is commonly used when copying or serializing a tree structure.
- **In-order (Left → Root → Right):** The left subtree is visited first, followed by the current node and then the right subtree. For a Binary Search Tree, In-order traversal always produces values in ascending order. This traversal applies only to Binary Trees and their derivatives. It is not defined for General Trees.
- **Post-order (Left → Right → Root):** Both subtrees are visited before the current node. This traversal is often used when a node should be processed after all of its descendants.
- **Breadth-First Search (BFS) – Level by Level:** Nodes are visited level by level from top to bottom using a queue. This traversal is supported across all five tree types in this project.

3. Core technologies

The project is a Maven-based JavaFX desktop application developed to visualize and demonstrate the operations of tree data structures through interactive graphical animations. The core technologies and tools used in the project are summarized below.

Technology Category	Technology Stack	Usage in Project
Programming language	Java 21	Core language used to build the logic, data structures, algorithms, and overall architecture of the application.
GUI Framework	JavaFX	The primary UI framework used to render the desktop graphical interface and UI animations.
Build Tool	Apache Maven	Used for dependency management, compiling the source code, and automating the project build process.
Layout Definition	FXML	An XML-based markup language used to structurally design and separate the UI layouts from the backend logic.
Styling	CSS	Used to customize the visual appearance of the application, including the implementation of Light/Dark modes.
Version Control	Git & GitHub	Used for source code management, tracking changes, and facilitating collaboration among team members.

4. Requirement analysis

4.1. Core Logic:

- The application models five fundamental tree data structures: General Tree, Binary Tree, Binary Search Tree (BST), AVL Tree, and Red-Black Tree.
- Each tree must support core operations including creating a root, inserting nodes, deleting nodes, searching for values, and updating existing node values.
- The system enforces specific algorithmic rules for each tree type, such as BST ordering, AVL height balancing and rotations, and Red-Black Tree recoloring and fix-ups.
- The application supports multiple traversal strategies: Pre-order, In-order, Post-order, and Breadth-First Search (BFS), returning the sequence of visited values.
- Operation history is maintained via a dual-stack system, allowing users to safely undo and redo structural changes.

4.2. Interactions:

- User input actions (insert, delete, update, search) trigger step-by-step algorithm executions in the backend model.
- The model emits semantic operation steps (e.g., comparing, rotating, recoloring, moving) to the visualization layer.
- The visualization layer converts these semantic steps into synchronized graphical animations, illustrating how the tree structure changes.
- A synchronized pseudocode panel tracks the execution flow and highlights the exact algorithmic step corresponding to the active animation frame.

4.3. User Interface:

- The main workspace features an interactive canvas that visually renders nodes and edges.
- Real-time tree statistics, including tree type, height, total number of nodes, and root value, are displayed and updated automatically after each operation.
- Users can fully control the animation playback through dedicated buttons (Play, Pause, Undo, Redo) and adjust the animation speed using a slider.
- The canvas supports mouse interactions, allowing users to zoom (scroll) and pan (drag) to navigate large or deeply unbalanced trees.

- The interface includes a global settings menu to switch between Light and Dark themes, as well as Help and Credit screens.

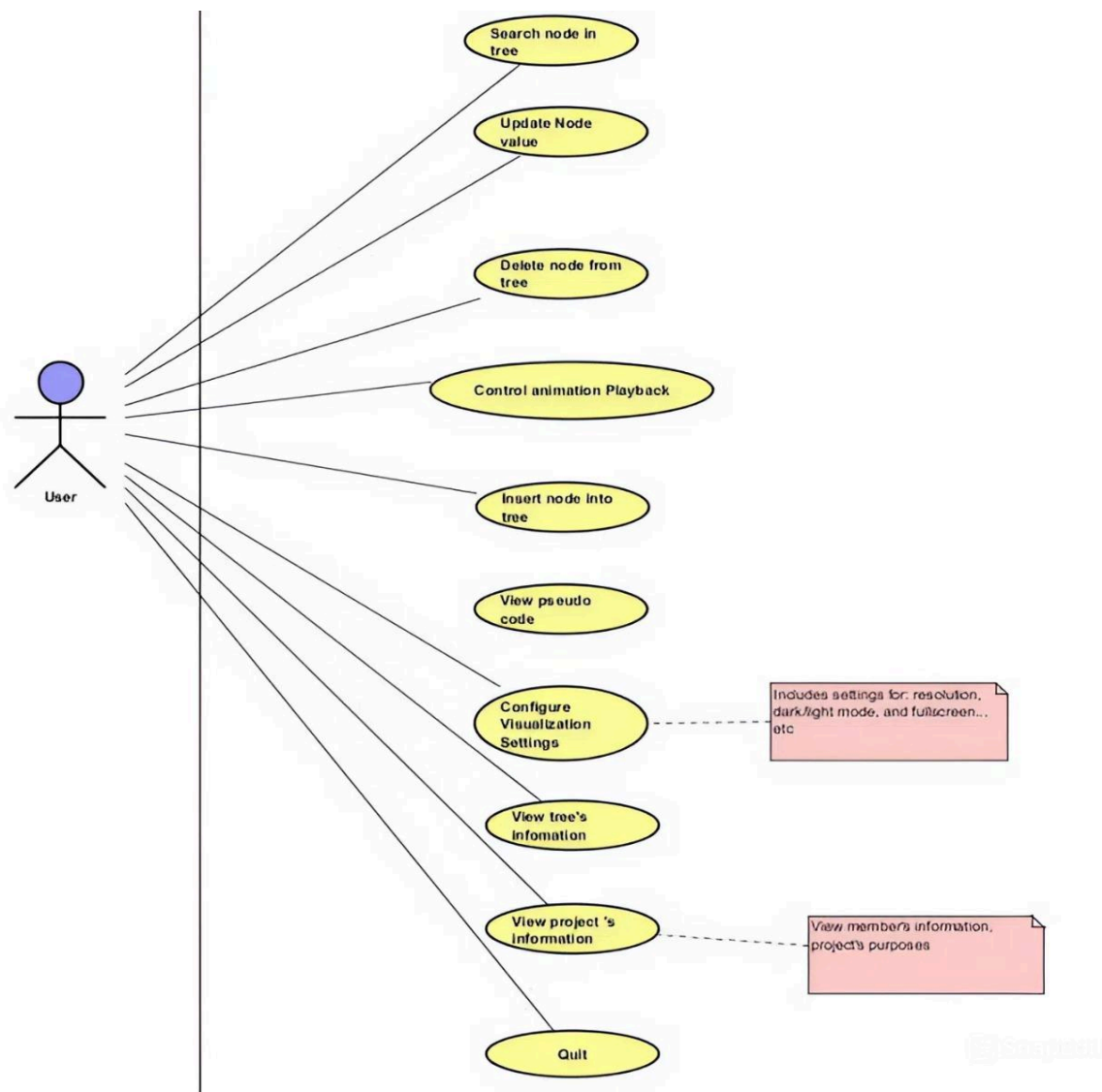
4.4. Object Oriented Programming requirements:

- Use classes to model tree data structures, tree nodes, operation histories, and visual UI elements.
- Inheritance and polymorphism are applied to share common tree behaviors (via an AbstractTree base class) while handling specific implementations for different tree algorithms.
- Encapsulation is strictly enforced to protect the internal state of trees and nodes, ensuring they are only modified through designated public operations.
- The Model-View-Controller (MVC) architectural pattern must be strictly followed to separate the pure algorithm logic from graphical rendering.

5. System design & MVC architecture

5.1. Use case modelling

This section describes the interactions between the system and its users, outlining the core functionalities available in the Tree Operation application.



Use case diagram

5.1.1. System Actor

The main actor interacting with the system is the User. The User operates the program through the user interface (UI) by clicking buttons, entering integer values, and selecting available functions.

5.1.2. General Functions

Before starting the visualization, the User can open the project's information to view team members and project purposes, or configure visualization settings such as screen resolution and light/dark themes. After that, the User can:

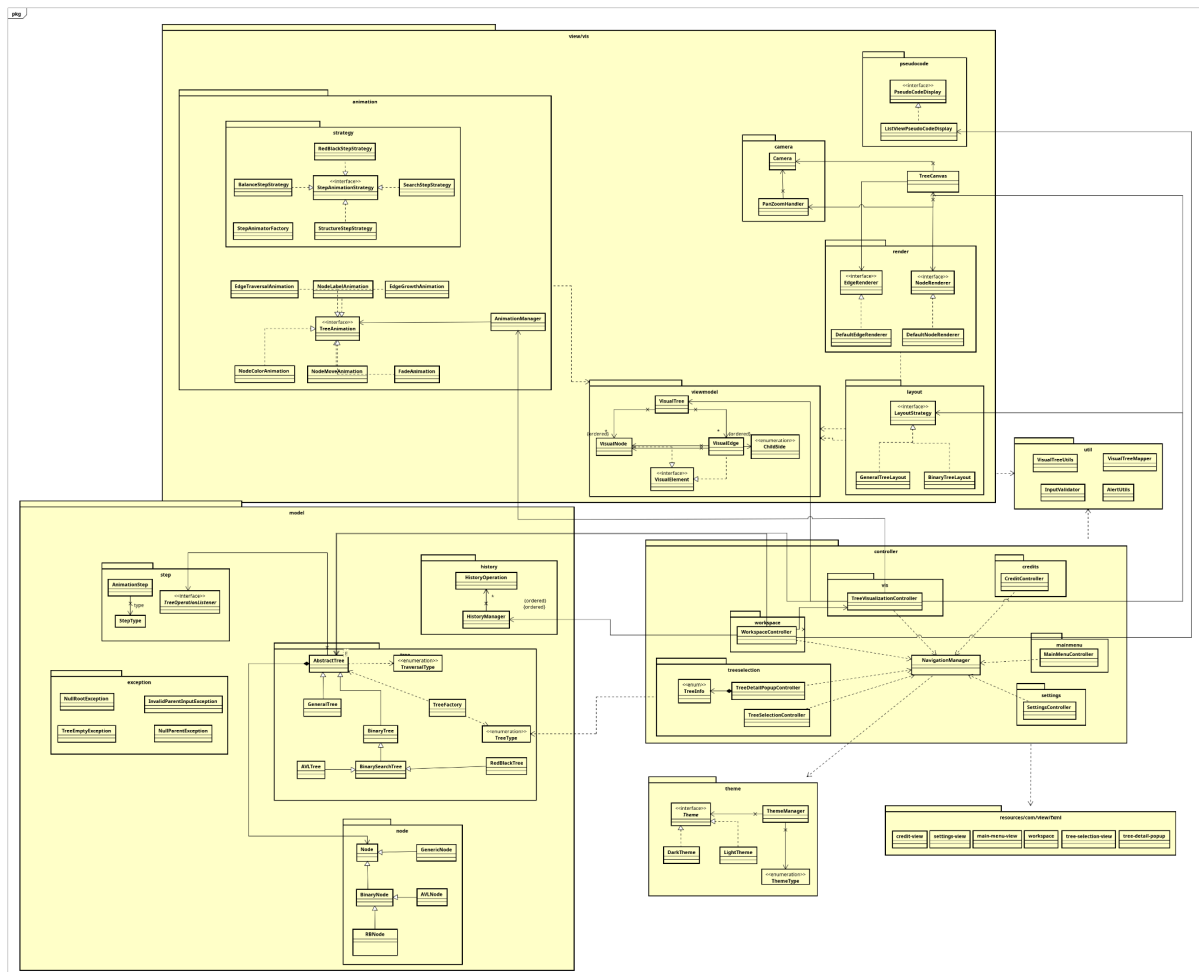
- Select a specific tree data structure to visualize.
- Execute core tree operations such as inserting, deleting, updating, or searching for a node.

- View real-time tree information and synchronize the algorithm's execution with a pseudo-code display.
- Control animation playback by pausing, playing, or adjusting the animation speed. When the User wants to end the program, they can click the Quit button on the interface to leave the system.

5.2 Class diagram

This section presents the class design of the Tree Operation application. The class diagrams are organized into two levels. The first level gives a general view of the whole system and shows the relationships among the main packages. The second level presents detailed diagrams for the core MVC packages and supporting packages used by the application.

5.2.1 General Class Diagram



The general class diagram shows that the application is organized based on the Model-View-Controller architecture. The Model package contains the logical tree structures and algorithms, including GeneralTree, BinaryTree, BinarySearchTree, AVLTree and RedBlackTree. These classes are built around the abstract base class AbstractTree, which defines common tree operations such as create, insert, delete, update, search, traverse, getHeight and getNumberOfNodes.

The View package is responsible for visual representation rather than tree logic. It contains visual models such as VisualTree, VisualNode and VisualEdge, rendering classes, layout strategies, animation classes, camera handling and pseudo-code display components. The Controller package connects JavaFX user actions to the model and view layers. For example,

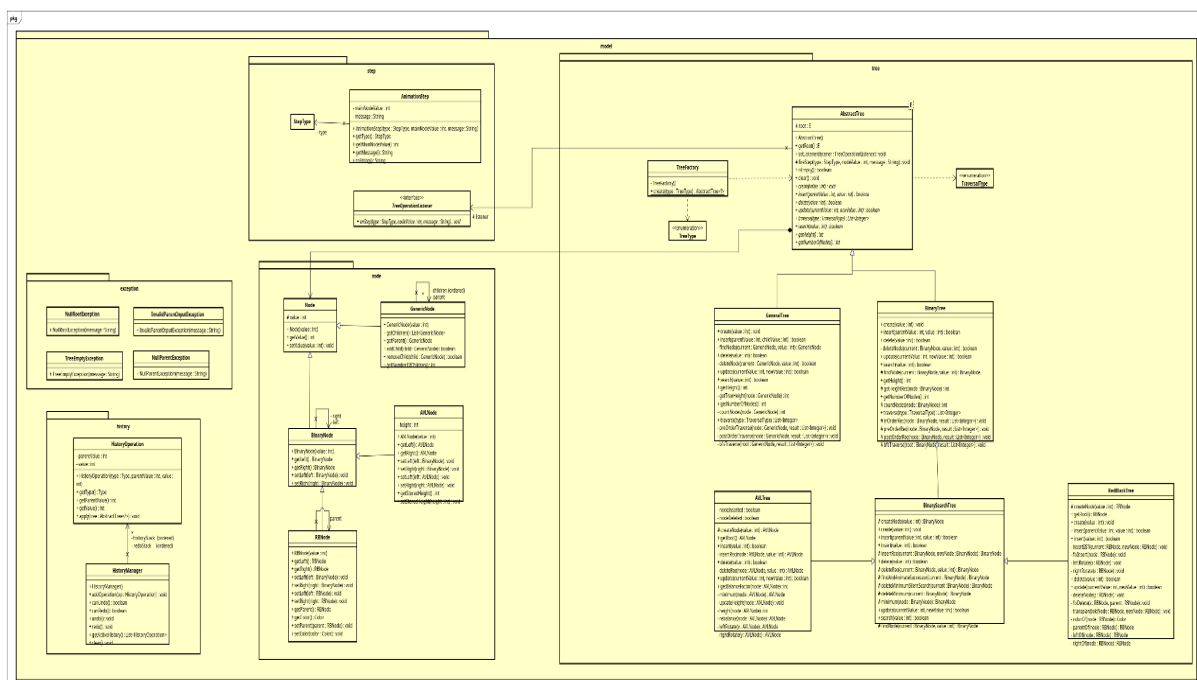
WorkspaceController receives user input, validates it, calls operations on the selected tree, and delegates visualization tasks to TreeVisualizationController.

A key design point in the diagram is the use of polymorphism. The controller works with `AbstractTree<?>` instead of depending on one concrete tree type. Therefore, the same operation buttons can call insert, delete, search or traverse on different tree implementations, while the actual behavior is decided at runtime. The same idea is also used in the view layer through interfaces such as `LayoutStrategy`, `NodeRenderer`, `EdgeRenderer`, `TreeAnimation`, `StepAnimationStrategy`, `PseudoCodeDisplay` and `Theme`.

The Model does not directly draw on the screen. Instead, tree algorithms emit operation steps through the `TreeOperationListener` interface. `TreeVisualizationController` receives these steps, stores them as `AnimationStep` objects, and converts them into visual animations. This keeps tree algorithms independent from JavaFX rendering and supports MVC separation clearly.

The diagram also includes supporting packages such as theme, util, history, step, exception and FXML resources. These packages help manage application appearance, input validation, tree mapping, undo/redo operations, algorithm step communication, error handling and screen layout resources.

5.2.2 Model class diagram



In the Model package, `AbstractTree` acts as the abstract base class for tree implementations. It defines the shared contract for tree operations, while concrete subclasses implement their own rules. `GeneralTree` and `BinaryTree` inherit directly from `AbstractTree`. `BinarySearchTree` extends `BinaryTree` by adding ordering rules. `AVLTree` and `RedBlackTree` extend

BinarySearchTree and override some behaviors to maintain balancing rules after insertion, deletion or update.

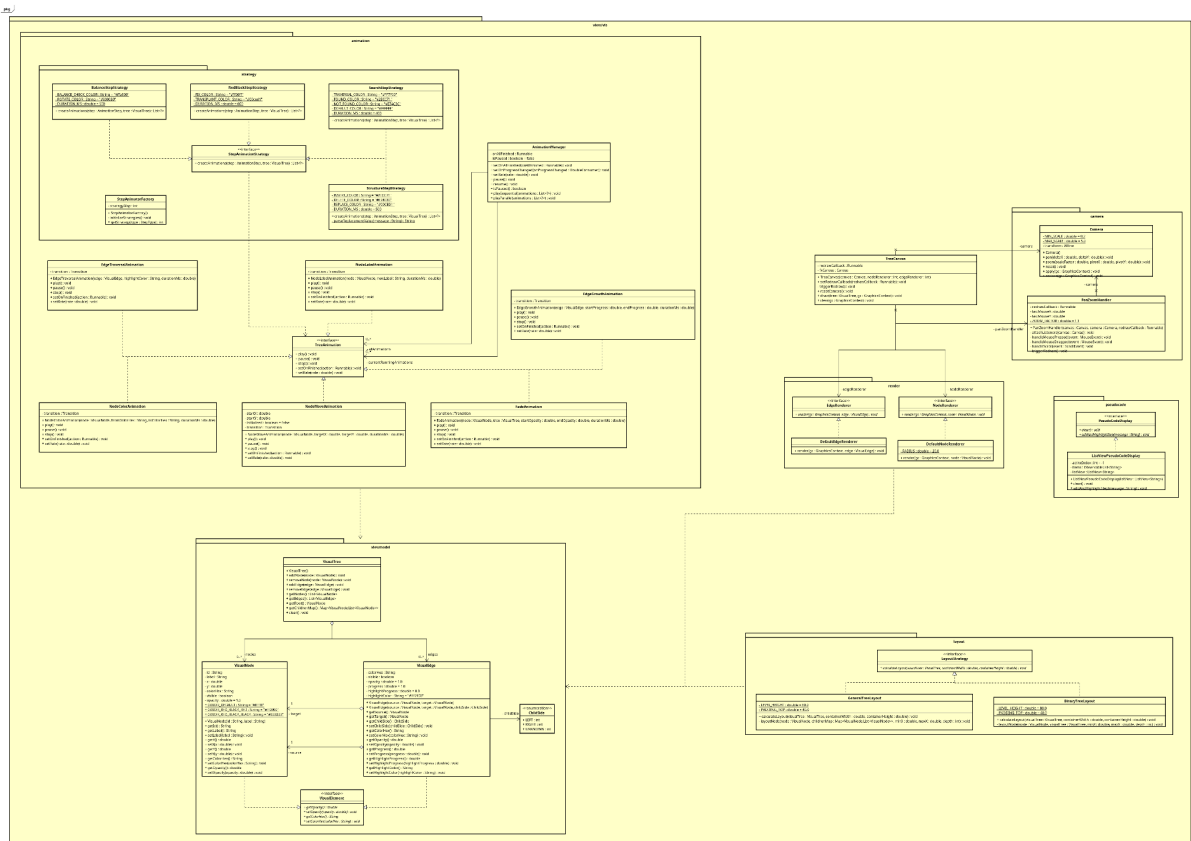
This design demonstrates abstraction, inheritance and polymorphism. WorkspaceController stores the current logical tree as AbstractTree<?>. Because of this, the controller can call the same methods on every tree type, but each subclass provides its own implementation. For example, insert behaves differently in GeneralTree, BinaryTree, BinarySearchTree, AVLTree and RedBlackTree. For self-balancing trees, AVLTree performs height updates and rotations, while RedBlackTree performs recoloring and fix-up operations.

The node hierarchy also shows inheritance and specialization. Node stores the common value field. GenericNode extends it with a list of children and a parent reference, which is suitable for general trees. BinaryNode extends it with left and right child references. AVLNode extends BinaryNode with a stored height field, while RBNode extends BinaryNode with parent and color information. Composition is used because tree objects contain root nodes, and nodes contain references to their child nodes.

The Model package also contains several supporting components. TreeFactory creates the correct tree object based on the selected TreeType, which helps the controller avoid directly instantiating concrete tree classes. TraversalType defines traversal modes used by tree traversal operations. The step package contains StepType, AnimationStep and TreeOperationListener, which allow model algorithms to report their execution steps without depending on the view layer. The history package contains HistoryManager and HistoryOperation, which support undo and redo operations. The exception package contains custom exceptions such as TreeEmptyException, NullRootException, NullParentException and InvalidParentInputException, making model-level error handling clearer and more meaningful.

Overall, the Model class diagram shows that the tree logic is separated from the user interface and visualization logic. The use of inheritance, abstraction, factory creation, operation history and listener-based step reporting makes the model layer reusable and easier to extend.

5.2.3 View class diagram



In the View package, the visualization layer is separated from the logical tree model. VisualTree is composed of VisualNode and VisualEdge objects. These classes represent the visual state of the tree instead of the actual data structure logic. VisualNode stores drawing-related information such as id, label, position, color and opacity. VisualEdge stores the connection between visual nodes, including source node, target node, opacity, progress, highlight progress and child side information.

The conversion from logical tree nodes to visual objects is handled by VisualTreeMapper, which is placed in the util package. This mapper builds or updates a VisualTree from an AbstractTree<?>. By separating mapping logic from rendering logic, the application keeps the model data structure independent from the canvas visualization.

The rendering part uses interfaces to reduce dependency on concrete drawing classes. TreeCanvas draws the current VisualTree by using NodeRenderer and EdgeRenderer. Their default implementations are DefaultNodeRenderer and DefaultEdgeRenderer. This design allows the canvas to render nodes and edges without knowing the detailed drawing rules. If the appearance of nodes or edges changes later, the renderer implementation can be changed without modifying the tree model.

The layout system also uses polymorphism through the LayoutStrategy interface. GeneralTreeLayout is used for general trees, while BinaryTreeLayout is used for binary-style trees. This allows the visualization controller to choose a layout algorithm based on the

selected tree type. As a result, the same visualization structure can support different kinds of trees.

The animation system is built around the `TreeAnimation` interface. Animation classes such as `NodeColorAnimation`, `NodeMoveAnimation`, `NodeLabelAnimation`, `FadeAnimation`, `EdgeGrowthAnimation` and `EdgeTraversalAnimation` implement this interface.

`AnimationManager` can play these different animations sequentially or in parallel through the same common interface.

The animation strategy classes apply the Strategy pattern. `StepAnimationStrategy` defines how an algorithm step should become visual effects. `SearchStepStrategy` handles traversal and search-related steps, `StructureStepStrategy` handles insert, delete and update changes, `BalanceStepStrategy` handles AVL balancing steps, and `RedBlackStepStrategy` handles recoloring and Red-Black tree fix-up steps. `StepAnimatorFactory` maps `StepType` values to the correct strategy. This design makes the system extensible because new step types or animation behaviors can be added without changing the model algorithms.

The View package also includes camera and pseudo-code display components. `Camera` and `PanZoomHandler` support canvas movement and zooming, helping users inspect large trees. `PseudoCodeDisplay` defines the interface for showing algorithm steps, and `ListViewPseudoCodeDisplay` implements it using a JavaFX `ListView`. These components improve the visualization experience while keeping the tree algorithms separate from the UI presentation.

5.2.4 Controller class diagram

TreeSelectionController manages the tree selection screen. It handles mouse events when the user selects General Tree, Binary Tree, Binary Search Tree, AVL Tree or Red-Black Tree. It

also opens `TreeDetailPopupController` to show information about the selected tree before starting visualization. `TreeDetailPopupController` uses the `TreeInfo` enum to store the selected tree's title and description, then sets `WorkspaceController.currentTreeType` and navigates to the workspace screen.

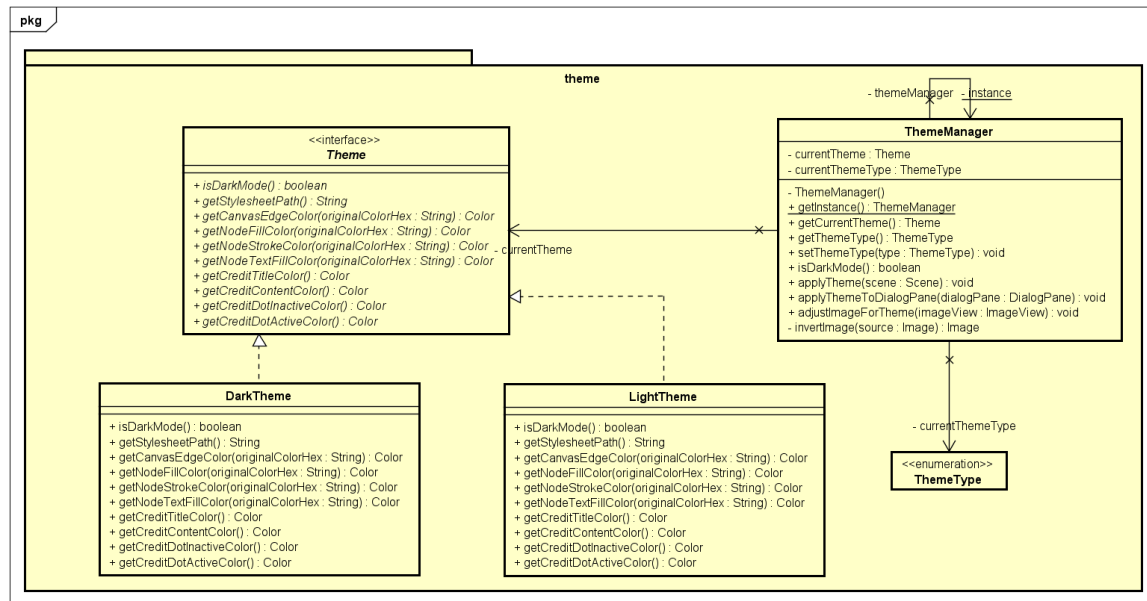
`WorkspaceController` is the main controller for tree operations. It stores the selected logical tree as `AbstractTree<?>`, which is an important use of polymorphism. Because the controller depends on the abstract type, the same UI buttons can execute operations on different concrete tree classes. For example, insert, delete, update, search and traverse are called through the `AbstractTree` interface, while the actual behavior is provided by `GeneralTree`, `BinaryTree`, `BinarySearchTree`, `AVLTree` or `RedBlackTree` at runtime.

`WorkspaceController` also coordinates input validation, operation execution, history management and UI updates. It uses `InputValidator` to parse user input, `AlertUtils` to display errors, `HistoryManager` to support undo and redo, and `TreeFactory` to create the correct tree type. After each operation, it updates statistics such as tree height, number of nodes, root value, balance factor and traversal result. Therefore, `WorkspaceController` acts as the main bridge between the user interface and the model.

`TreeVisualizationController` is a specialized controller for visualization. It implements `TreeOperationListener`, so it can receive algorithm steps emitted by the model. These steps are stored as `AnimationStep` objects, converted into `TreeAnimation` objects by `StepAnimatorFactory`, and then played by `AnimationManager`. This design separates user operation handling from animation handling: `WorkspaceController` focuses on user commands, while `TreeVisualizationController` focuses on rendering and animation flow.

The relationship between `WorkspaceController` and `TreeVisualizationController` also shows MVC separation. `WorkspaceController` calls tree operations and manages the workspace UI, but it does not directly draw nodes or edges. `TreeVisualizationController` receives model events, updates the `VisualTree`, applies a `LayoutStrategy` and asks `TreeCanvas` to render the result. Through this structure, the controller layer coordinates the system without mixing tree algorithms, UI event handling and low-level drawing code in one class.

5.2.5 Theme class diagram

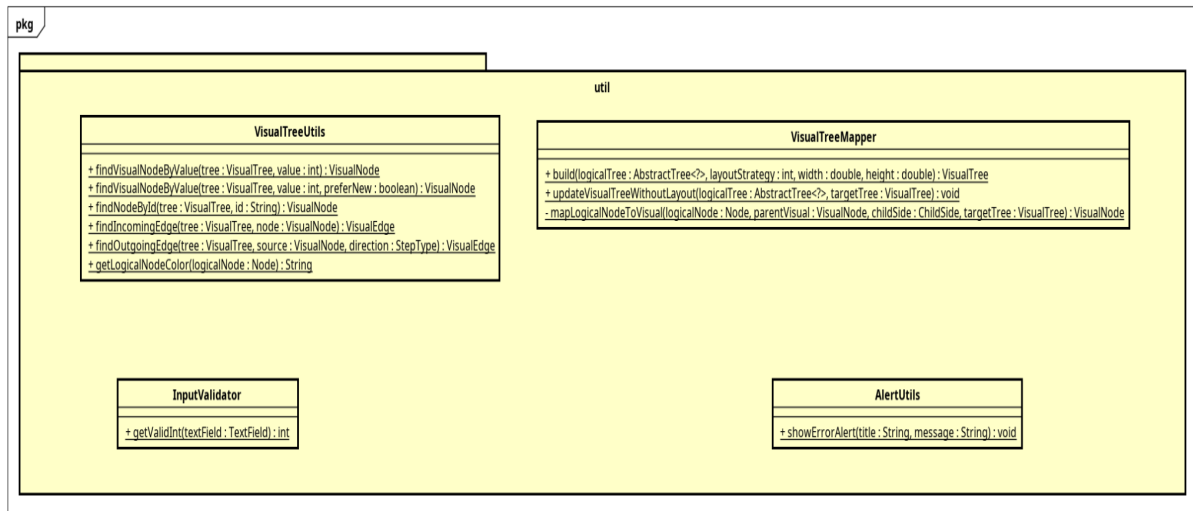


The Theme package supports the visual appearance of the application. Although it is not part of the tree algorithm, it helps keep the user interface consistent across screens, dialogs and canvas rendering. The package contains the Theme interface, two concrete theme classes, ThemeManager and the ThemeType enumeration.

The Theme interface defines common methods for theme-related behavior, such as checking dark mode, getting the stylesheet path and providing colors for canvas edges, node fill, node stroke, node text and credit page elements. LightTheme and DarkTheme implement this interface with different color rules. Because both classes follow the same interface, the application can switch between light and dark modes without changing the controller or rendering logic.

ThemeManager controls the active theme and stores the current ThemeType. It provides access to the current theme, applies the selected theme to JavaFX scenes and dialogs, and adjusts images when necessary. Since ThemeManager stores the active theme as a Theme, it can work with either LightTheme or DarkTheme polymorphically. This design centralizes theme handling and prevents color or stylesheet logic from being scattered across many classes.

5.2.6 Util class diagram



The Util package contains helper classes used by different parts of the application. These classes are not main business objects, but they reduce duplicated code and make controllers and visualization classes easier to maintain.

InputValidator validates and converts text field input into integer values before the controller sends data to the model. This helps prevent invalid input from breaking tree operations.

AlertUtils provides a reusable method for showing error alerts and applies the current theme to alert dialogs, keeping error messages visually consistent with the rest of the application.

VisualTreeMapper converts logical tree data into visual tree data. It builds a VisualTree from an AbstractTree<?> and a selected LayoutStrategy, then maps logical nodes into VisualNode objects and parent-child relationships into VisualEdge objects. This separates model data structures from canvas rendering.

VisualTreeUtils provides lookup methods used during visualization and animation. It can find visual nodes by value or id, find incoming or outgoing edges and get the display color of a logical node, especially for Red-Black tree nodes. Overall, the Util package supports validation, alert handling, logical-to-visual mapping and visualization lookup operations while keeping the main MVC classes focused on their own responsibilities.

5.3 MVC Architecture

The application follows the Model-View-Controller pattern to separate data, presentation and user interaction.

The **Model** layer contains the tree data structures and algorithms. Classes such as AbstractTree, GeneralTree, BinaryTree, BinarySearchTree, AVLTree and RedBlackTree are responsible for maintaining the logical tree state and performing tree operations. The model does not know how the tree is drawn on the screen. Instead, it reports meaningful algorithm events through the TreeOperationListener interface.

The **View** layer contains the visual representation of the tree and the user interface components. FXML files define the JavaFX screens, while classes such as `TreeCanvas`, `VisualTree`, `VisualNode`, `VisualEdge`, renderers, layout strategies and animation classes handle visualization. The view displays the result of operations but does not define tree operation rules.

The **Controller** layer connects the model and the view. `WorkspaceController` receives user actions from the interface, validates input, selects the correct tree operation and updates UI information. `TreeVisualizationController` receives operation steps from the model, stores them as `AnimationStep` objects and converts them into animations on the canvas. This makes the controller a bridge between logical operations and visual feedback.

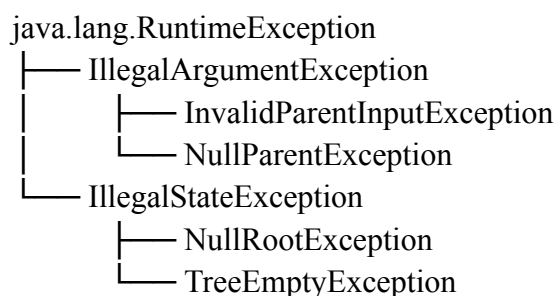
This MVC separation improves maintainability. Tree algorithms can be modified without rewriting the JavaFX interface, and visual rendering can be improved without changing model classes. It also supports polymorphism naturally because controllers depend on abstract types and interfaces, while concrete behavior is selected at runtime based on the chosen tree type, layout strategy, animation strategy, renderer or theme.

5.4. Exception Handling

To ensure application stability, separation of concerns, and a graceful user experience, the system implements a strict exception handling strategy. Errors are categorized into domain-specific custom exceptions representing invalid tree states and standard Java exceptions for general input and structural validation.

5.4.1. Custom Domain Exceptions

Custom exceptions are isolated within the **`com.model.exceptions`** package. They extend standard Java runtime exceptions to avoid polluting method signatures with **`throws`** clauses while providing precise contextual meaning for tree manipulation failures.



The custom hierarchy consists of the following definitions and allocation behaviors:

Exception Class	Base Class	Trigger Condition and Usage
<code>InvalidParentInputException</code>	<code>IllegalArgumentException</code>	<i>Reserved for future expansion.</i> Intended for operations where a parent node parameter is

		provided but strictly forbidden by the architecture.
<code>NullParentException</code>	<code>IllegalArgumentException</code>	Thrown when an insertion operation specifies a parent value that does not exist within a non-empty tree. Enforced in BinaryTree and GeneralTree .
<code>NullRootException</code>	<code>IllegalStateException</code>	Thrown when attempting to insert a child node relative to a parent, but the tree itself is empty (root is null). Enforced across BinarySearchTree , BinaryTree , GeneralTree , and RedBlackTree .
<code>TreeEmptyException</code>	<code>IllegalStateException</code>	Thrown when a traversal algorithm is invoked on a tree containing no nodes. Enforced in BinaryTree and GeneralTree .

5.4.2. Defensive Programming via Standard Exceptions

Standard Java exceptions are thrown explicitly throughout the codebase as a defensive programming measure to enforce type safety, structural integrity, and contract compliance before errors cascade to the visual layer.

- **Type Safety & Polimorphic Constraints:** Because specific tree implementations require specialized nodes, **AVLNode** and **RBNode** explicitly throw an **IllegalArgumentException** if an attempt is made to link a child node that is not an instance of their respective node classes.
- **Factory and Strategy Validation:** The **TreeFactory** validates tree construction requests, throwing an **IllegalArgumentException** if the tree type parameter is null or maps to an unsupported tree variation. Similarly, **BinaryTree** and **GeneralTree** throw this exception if a null traversal strategy is supplied.
- **UI Component and Graph Structural Constraints:** Within the visualization subsystem, **VisualEdge** throws an **IllegalArgumentException** if either its source or target visual nodes are initialized as null, ensuring the JavaFX scene graph cannot render a broken or disconnected edge.
- **Unsupported Tree Polymorphism:** **GeneralTree** explicitly throws an **UnsupportedOperationException** if an in-order traversal is requested, as this traversal mechanism is structurally undefined for non-binary trees.

5.4.3. UI Input Validation and JavaFX Integration

To protect the backend data models from dirty data, an explicit validation utility is executed prior to mutating tree states.

The **InputValidator** utility intercepts raw input from JavaFX UI components and converts implicit failures into predictable exceptions:

- **NullPointerException:** Triggered immediately if the monitored JavaFX TextField object reference itself is null, catching structural UI binding errors early.
- **NumberFormatException:** Thrown when a user attempts to submit an empty text field into a tree expecting numeric values.

Architectural Note on UI Propagation: These validation exceptions are designed to be caught upstream by the JavaFX controller layer. Rather than allowing a **NumberFormatException** to freeze the JavaFX Application Thread, the controller catches these runtime exceptions and maps them directly to non-blocking visual feedback (such as turning input borders red or presenting a user-friendly **Alert** dialog).

6. Demonstration & result

This section demonstrates the core functionalities of the Tree Data Structure Visualizer, showcasing the user interface, interactive features, and the overall results achieved through the architectural design.

6.1. System Overview and Navigation

The application provides a seamless and intuitive user flow. Upon launching the software, users are greeted with a centralized Main Menu. From there, navigating to the **Tree Selection Interface** allows users to choose from various supported data structures, including AVL Tree, Binary Tree, Red-Black Tree, Binary Search Tree, and General Tree.

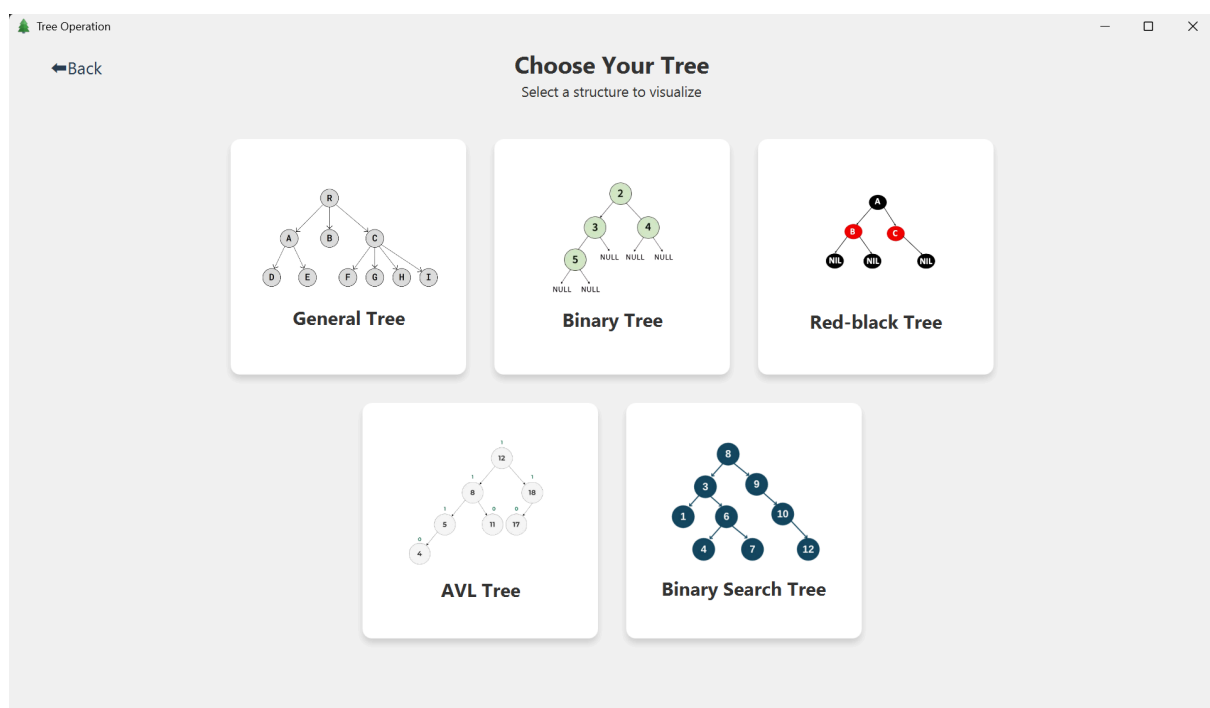


Figure 4.1: The Tree Selection interface displaying available tree data structures.

6.2. Core Functionalities Demonstration

The core of the application resides in the **Workspace**, a highly interactive laboratory where users can visually construct and manipulate tree structures.

6.2.1. Dynamic Tree Operations (Insert, Delete, Search)

The visualizer dynamically reacts to user inputs. When an operation such as an insertion or deletion is triggered, the engine calculates the new logical state and seamlessly animates the transition. Nodes physically translate across the screen to accommodate new branches, and

specific tree rules (such as AVL balancing or Red-Black color flipping) are animated in real-time. During a Search operation, the traversed path is vividly highlighted to guide the user's focus.

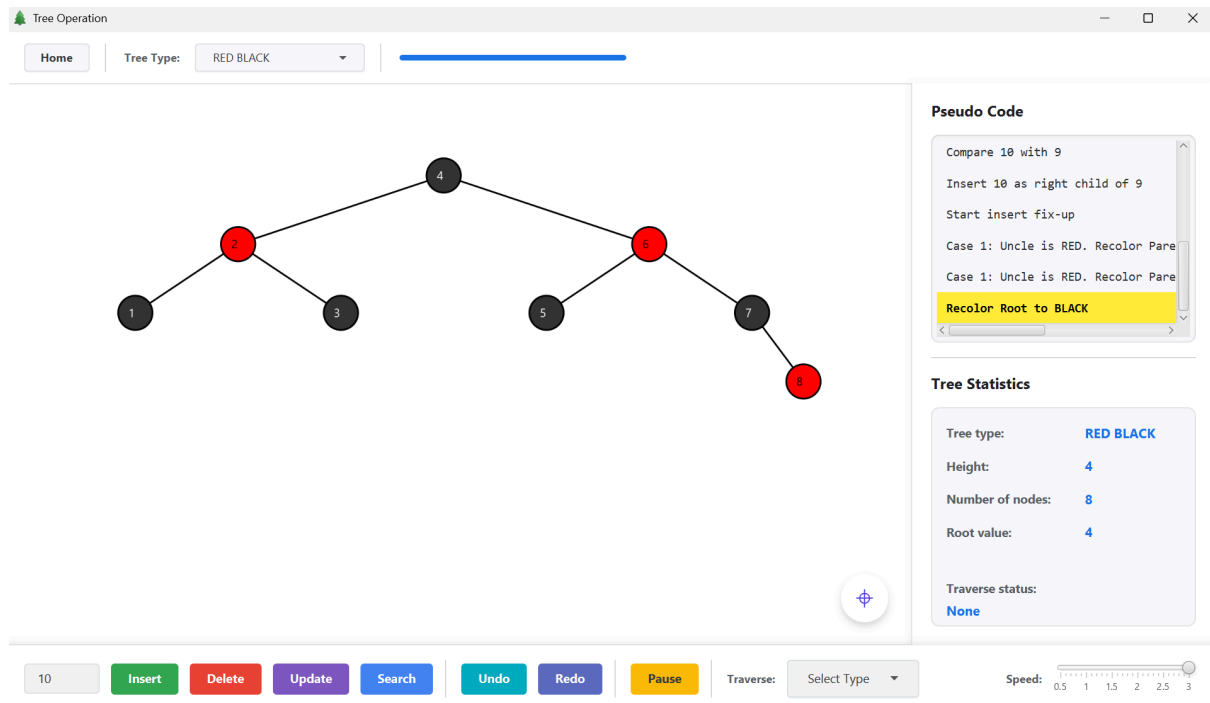


Figure 4.2: A fully constructed Red-Black Tree demonstrating the accurate layout algorithm.

6.2.2. Algorithmic Tracking via Pseudo-code

To maximize educational value, the application features a synchronized **Pseudo-code Panel**. As the tree animation progresses step-by-step, the corresponding line of pseudo-code is highlighted. This bridges the gap between abstract visual movements and concrete programming logic.

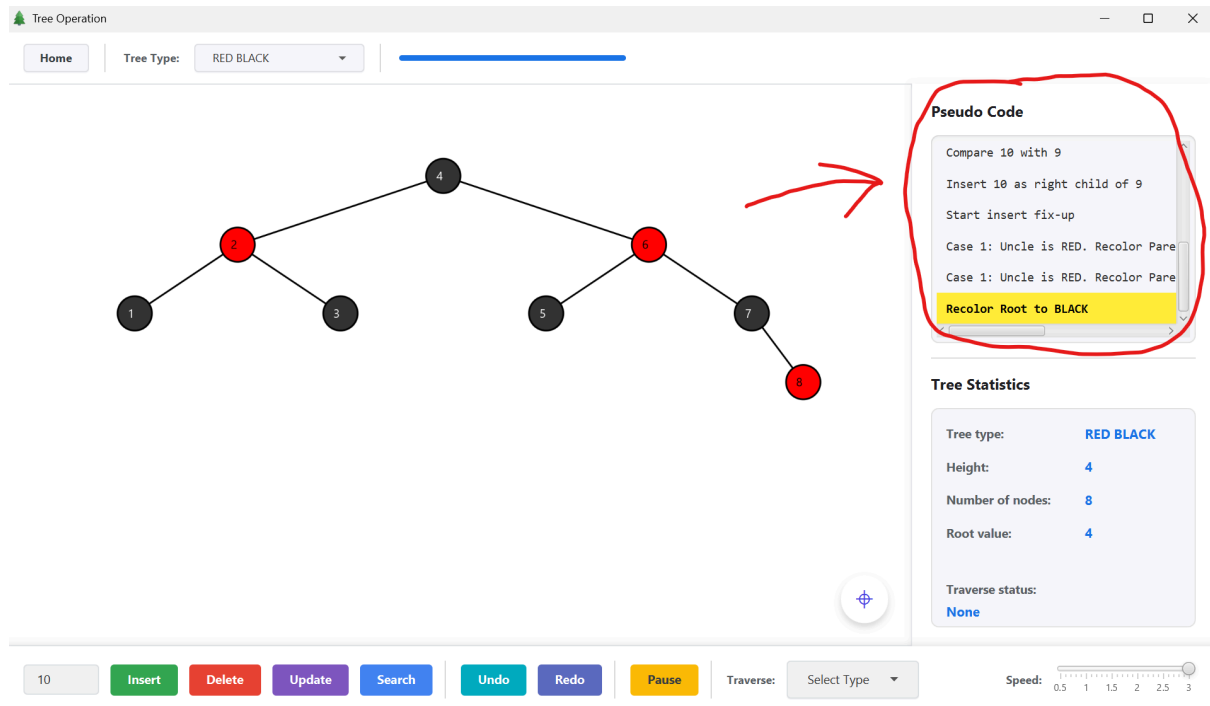


Figure 4.3: Synchronized pseudo-code tracking during a tree operation.

6.2.3. Tree Traversals

The system supports multiple traversal algorithms (Pre-order, In-order, Post-order, and Level-order). When a traversal is initiated, the visualizer sequentially illuminates the nodes, allowing users to visually comprehend the recursive patterns of different traversal techniques.

6.3. Advanced GUI Features

Beyond basic operations, several advanced features were implemented to enhance usability and provide a premium user experience.

6.3.1. Infinite Canvas and Camera Controls

To accommodate massively scaled trees, the rendering engine is built on an infinite canvas. Users can freely **Pan** (click and drag) and **Zoom** (scroll wheel) to explore different branches of large trees. A dedicated **Recenter** (⊕) button instantly resets the camera view back to the root node.

6.3.2. Undo/Redo & History Management

A highly robust **Undo/Redo** system was engineered using the Event-Sourcing pattern. Instead of storing heavy memory snapshots, the system records operational commands. Users can instantly traverse backward and forward through their operation history, with the rendering engine effortlessly reconstructing and animating the tree's prior states.

6.3.3. Premium Dark/Light Themes

The application features a modern UI design with dynamic theme switching. The **Premium Dark Mode** utilizes a curated "Slate & Indigo" color palette with deep dropshadows and neon-glow hover effects, offering a highly professional aesthetic akin to modern IDEs.

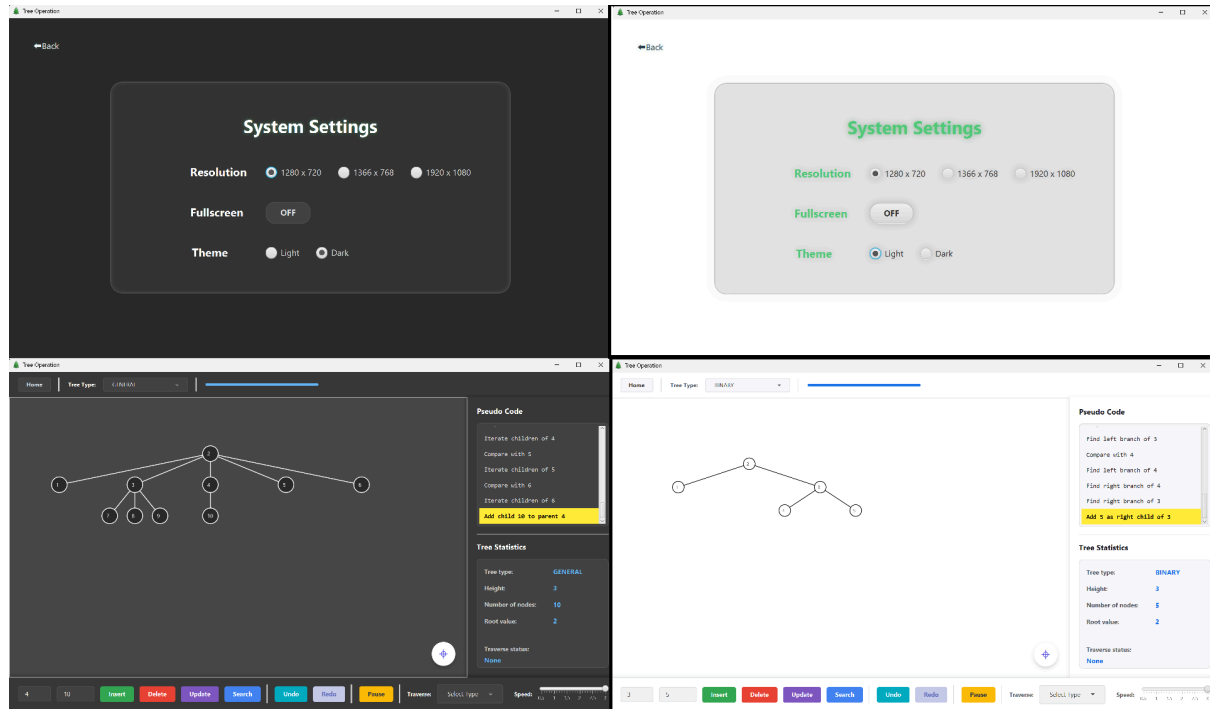


Figure 4.4: UI comparison demonstrating the dynamic Dark/Light theme integration.

6.4. Evaluation and Results

The final software successfully meets and exceeds the initial project requirements:

1. **Performance:** The custom rendering loop, utilizing JavaFX AnimationTimer, consistently outputs smooth **60-FPS animations**. Complex layout recalculations for large trees execute in microseconds without blocking the UI thread.
2. **Architectural Integrity:** By strictly adhering to **SOLID principles** and the **MVC pattern**, the complex tree algorithms are completely decoupled from the rendering layer. This ensures the codebase remains maintainable and highly extensible for future updates.
3. **Educational Impact:** The combination of step-by-step animations, real-time tree statistics, and synchronized pseudo-code successfully delivers a powerful educational tool for computer science students to study data structures.

7. Conclusion

This project successfully developed an interactive JavaFX-based application for visualizing tree data structures and their associated algorithms. The system supports five fundamental

tree types, including General Tree, Binary Tree, Binary Search Tree, AVL Tree, and Red-Black Tree, together with essential operations such as insertion, deletion, searching, updating, and traversal. Through real-time animations and synchronized pseudocode visualization, users can observe both the structural changes within a tree and the underlying algorithmic processes. The project also demonstrates the effective application of Object-Oriented Programming principles and the Model-View-Controller architecture to create a modular, maintainable, and extensible software system.

Despite its achievements, the project has several limitations. The application currently supports only integer values as node data, which restricts its flexibility for more general use cases. In addition, the tree layout algorithm may produce overlapping nodes or uneven spacing when visualizing very large or highly unbalanced trees. Furthermore, the system does not currently provide file persistence functionality, preventing users from saving and reloading their tree configurations between sessions.

Several opportunities exist for future improvement. Additional data structures such as B-Trees and Heaps could be incorporated to broaden the educational scope of the application. Support for generic data types would increase versatility, while file import/export functionality would enhance usability. Furthermore, developing a web-based version of the system could improve accessibility and allow users to interact with the application without requiring local installation.